

COM 3240

REINFORCEMENT LEARNING



Create me a painting with
the title artificial neural
networks in the style of Dali

DALL·E

FUTURE REWARDS

THE Q-LEARNING ALGORITHM

1. Initialise $Q(s, a)$ arbitrarily for all $s \in S$ and $a \in A(s)$.
2. Repeat (for each episode):
 - a. Initialise s .
 - b. Repeat (for each step of episode):
 - i. Choose an action a from s using a policy derived from Q (e.g., ϵ -greedy).
 - ii. Take action a , observe reward r and next state s' .
 - iii. $Q(s, a) \leftarrow Q(s, a) + \eta * [r + \gamma * \max_{a'} Q(s', a') - Q(s, a)]$.
 - iv. $s \leftarrow s'$.
 - c. until s is terminal.

Off-policy



Ramon y Cahal

1mm³: 10,000 neurons 3km wires

ARTIFICIAL NEURAL NETWORKS AS FUNCTION APPROXIMATIONS

SINGLE NEURON

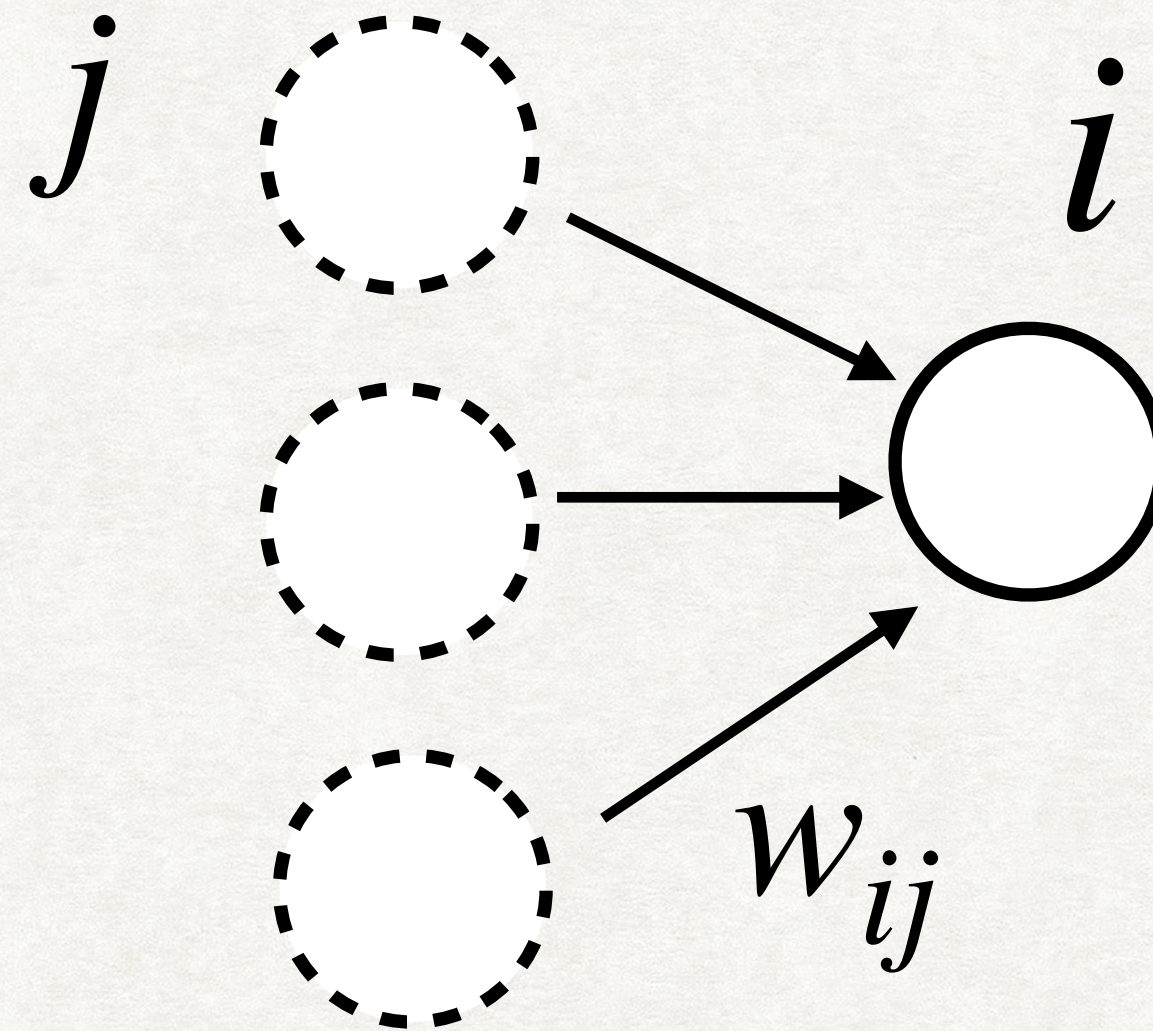
$$\mathbf{y} = F(\mathbf{x})$$

Samples from the input and output space of the function.

$$\mathbf{x}^{(1)} \quad \mathbf{x}^{(2)} \quad \mathbf{x}^{(3)} \dots \mathbf{x}^{(n)}$$

$$\mathbf{y}^{(1)} \quad \mathbf{y}^{(2)} \quad \mathbf{y}^{(3)} \dots \mathbf{y}^{(n)}$$

$$\hat{\mathbf{y}} = \hat{F}(\mathbf{x})$$



$$E = \frac{1}{2n} \sum_i \sum_{\mathbf{x}} \left(y_i^t(\mathbf{x}) - y_i(\mathbf{x}; \mathbf{W}) \right)^2$$

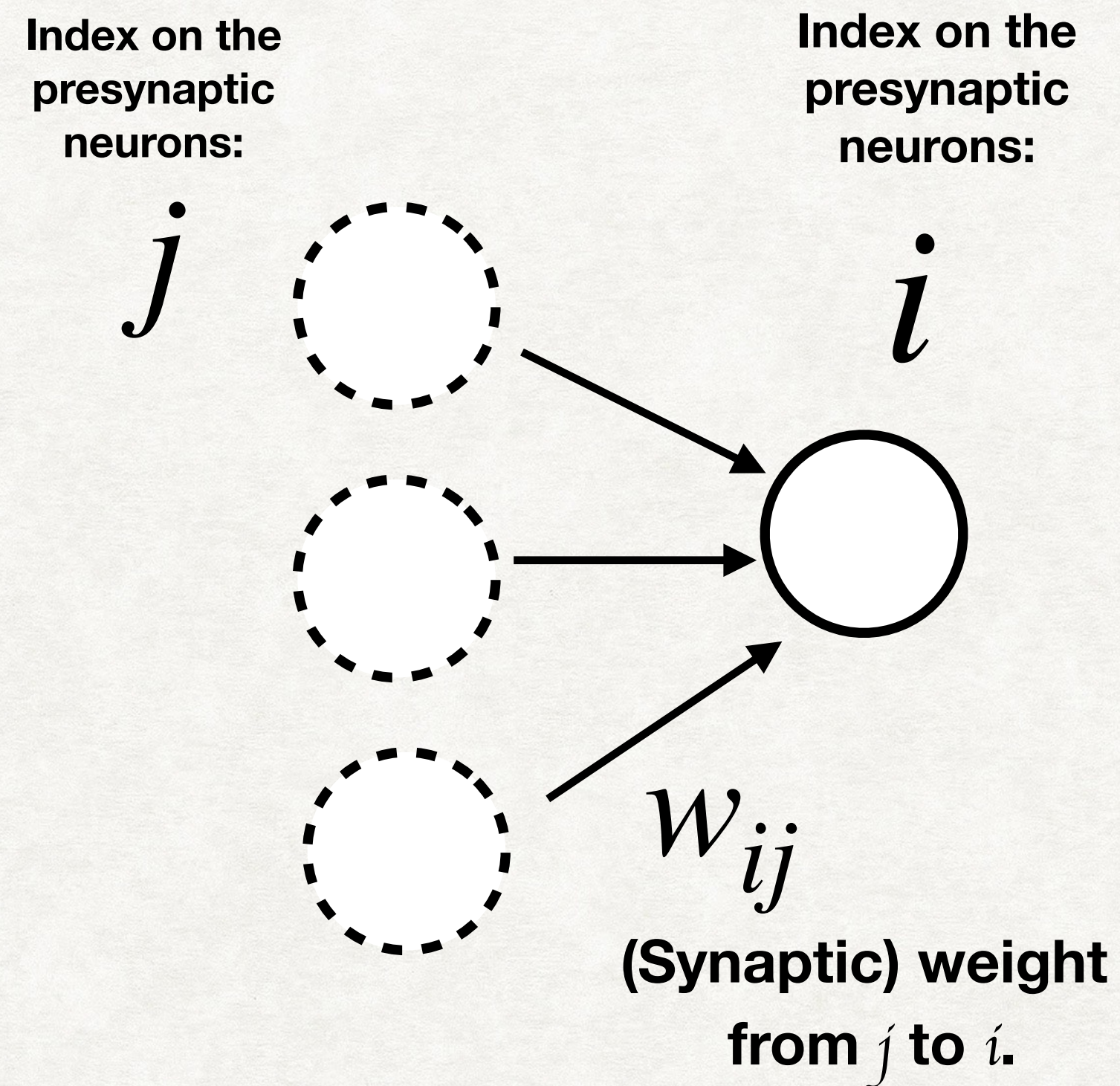
Find parameters (weights) that minimise the Loss function.

$$\Delta \mathbf{W}_{lk} \propto - \frac{\partial E}{\partial \mathbf{W}_{lk}}$$

$$\Delta \mathbf{W} \propto - \nabla_{\mathbf{W}} E$$

ARTIFICIAL NEURAL NETWORKS AS FUNCTION APPROXIMATIONS

SINGLE NEURON



$$y = ax + b$$

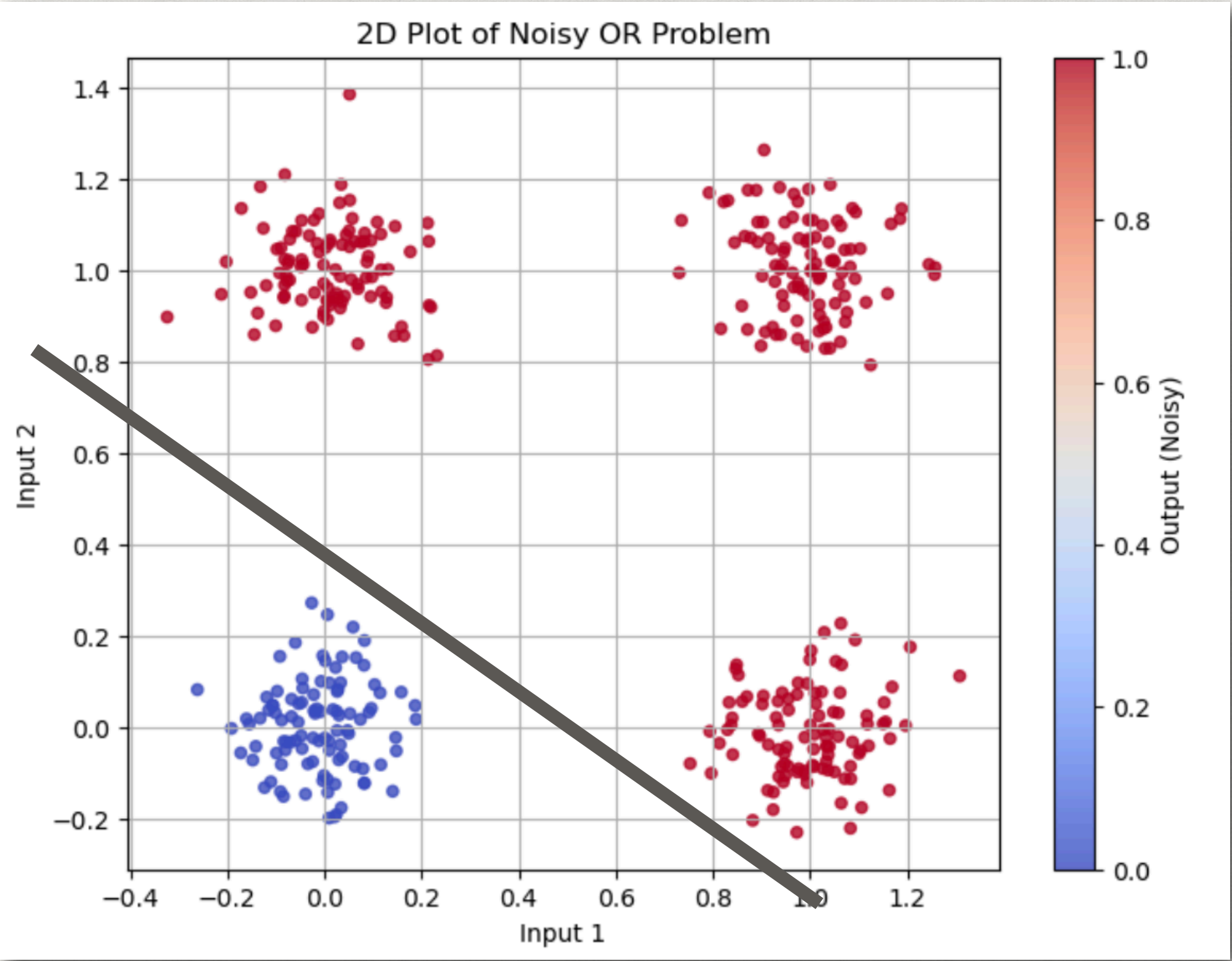
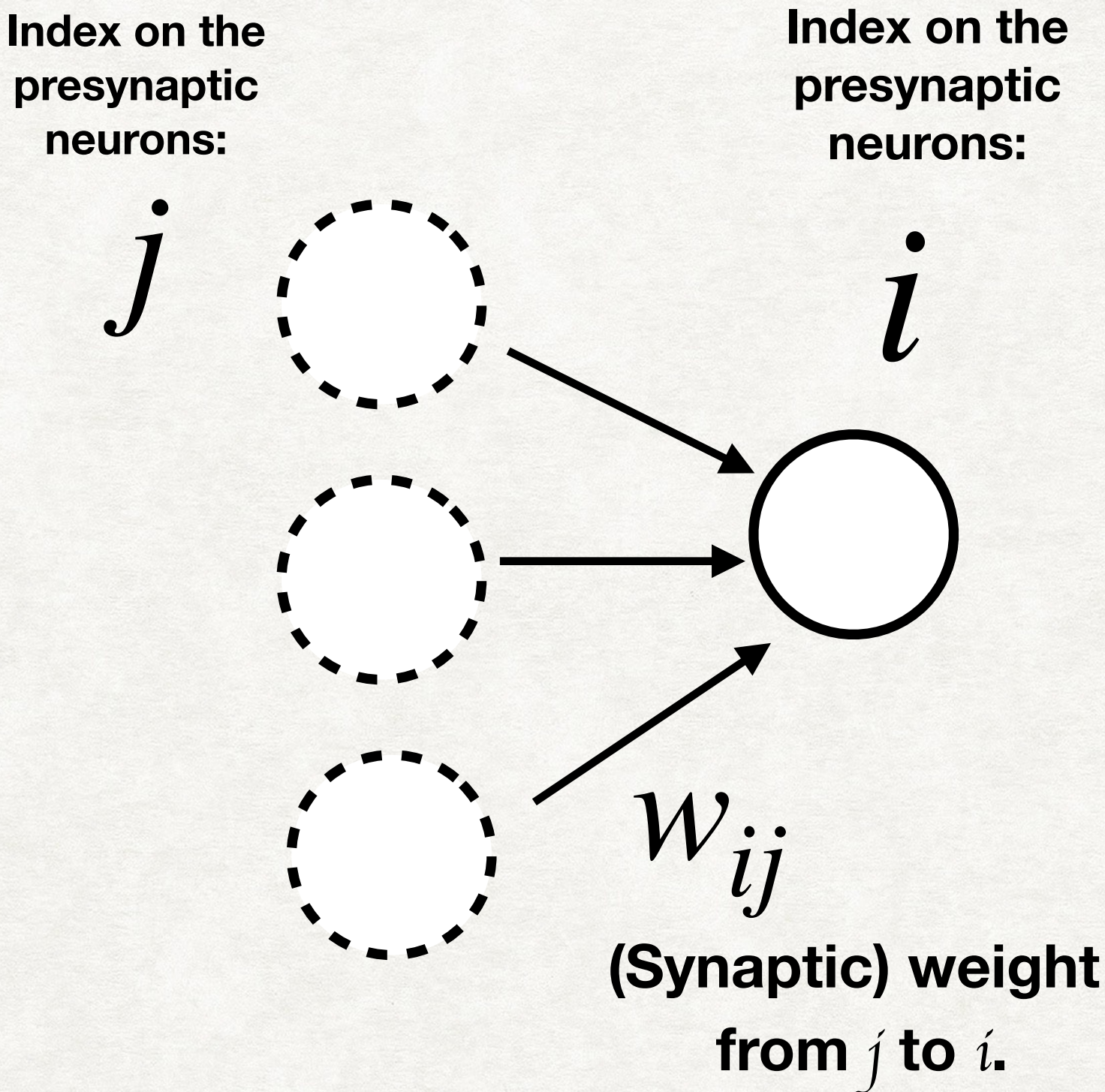
$$h_i = \sum_j w_{ij} x_j + b_i$$

$$x_i = f(h_i)$$

Activation function f , sigmoidal or ReLU

ARTIFICIAL NEURAL NETWORKS AS FUNCTION APPROXIMATIONS

LINEAR SEPARABILITY



Single layer

ARTIFICIAL NEURAL NETWORKS AS FUNCTION APPROXIMATIONS

GRADIENT DESCENT - BACKPROPAGATION ALGORITHM

$$E = \frac{1}{2n} \sum_i \sum_{\mathbf{x}} \left(y_i^t(\mathbf{x}) - y_i(\mathbf{x}; \mathbf{W}) \right)^2$$

$$\Delta w_{ij} = -\eta \frac{\partial E}{\partial w_{ij}}$$

ARTIFICIAL NEURAL NETWORKS AS FUNCTION APPROXIMATIONS

BACKPROPAGATION ALGORITHM FOR ONE LAYER

$$\Delta w_{ij} = -\eta \frac{1}{n} \sum_{\mathbf{x}} \left(y_i(\mathbf{x}) - y_i^t(\mathbf{x}) \right) y_i(\mathbf{x})(1 - y_i(\mathbf{x})) x_j$$

$$\Delta b_i = -\eta \frac{1}{n} \sum_{\mathbf{x}} \left(y_i(\mathbf{x}) - y_i^t(\mathbf{x}) \right) y_i(\mathbf{x})(1 - y_i(\mathbf{x}))$$

f: sigmoid

ARTIFICIAL NEURAL NETWORKS AS FUNCTION APPROXIMATIONS

GRADIENT DESCENT - BACKPROPAGATION ALGORITHM

$$\Delta w_{ij} = \eta \frac{1}{n} \sum_{\mathbf{x}} (y_i^t(\mathbf{x}) - y_i(\mathbf{x})) \cdot \mathbf{ReLU}' \left(\sum_j w_{ij} x_j + b_i \right) \cdot x_j$$

$$\Delta b_i = \eta \frac{1}{n} \sum_{\mathbf{x}} (y_i^t(\mathbf{x}) - y_i(\mathbf{x})) \cdot \mathbf{ReLU}' \left(\sum_j w_{ij} x_j + b_i \right)$$

derivative of rectified linear unit: $\mathbf{ReLU}'$

f : $\mathbf{ReLU}$

ARTIFICIAL NEURAL NETWORKS AS FUNCTION APPROXIMATIONS

EFFICIENT IMPLEMENTATION

$$y_i(\mathbf{x}) = f \left(\sum_j w_{ij} x_j + b_i \right)$$

X = ... # Input data matrix, shape (batch_size, input_dim)
y_t = ... # Target values matrix, shape (batch_size, output_dim)
y = ... # Computed outputs matrix, shape (batch_size, output_dim)
W = ... # Weight matrix, shape (input_dim, output_dim)
b = ... # Bias vector, shape (output_dim,)
learning_rate = ... # Learning rate, a scalar

Always check DIMENSIONS

ARTIFICIAL NEURAL NETWORKS AS FUNCTION APPROXIMATIONS

EFFICIENT IMPLEMENTATION

$$y_i(\mathbf{x}) = f \left(\sum_j w_{ij} x_j + b_i \right)$$

Code Snippet:

Compute the linear transformation using matrix multiplication

linear_output = np.matmul(X, W) + b

Apply activation function element-wise

y_i = activation_function(linear_output)

ARTIFICIAL NEURAL NETWORKS AS FUNCTION APPROXIMATIONS

EFFICIENT IMPLEMENTATION

$$\Delta w_{ij} = -\eta \frac{1}{n} \frac{\partial E}{\partial w_{ij}} = -\eta \frac{1}{n} \sum_{\mathbf{x}} (y_i(\mathbf{x}) - y_i^t(\mathbf{x})) y_i(\mathbf{x})(1 - y_i(\mathbf{x})) x_j,$$

Code Snippet:

Compute updates for weights

```
update_weights = -learning_rate * np.matmul(X.T, error_times_derivative) /  
X.shape[0]
```

f: sigmoid

ARTIFICIAL NEURAL NETWORKS AS FUNCTION APPROXIMATIONS

EFFICIENT IMPLEMENTATION

$$\Delta b_i = -\eta \frac{1}{n} \sum_{\mathbf{x}} (y_i(\mathbf{x}) - y_i^t(\mathbf{x})) y_i(\mathbf{x})(1 - y_i(\mathbf{x}))$$

Code Snippet:

Compute updates for biases

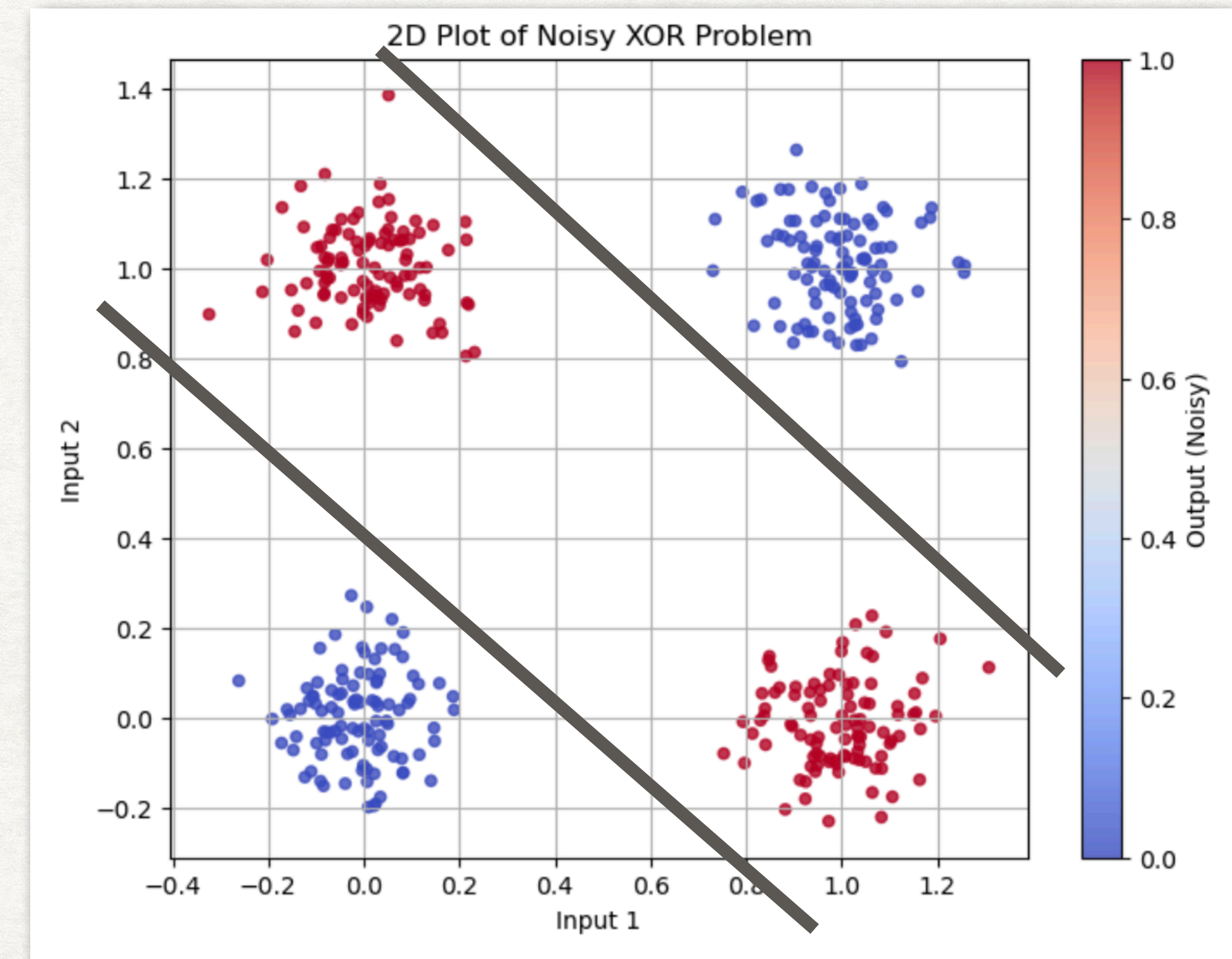
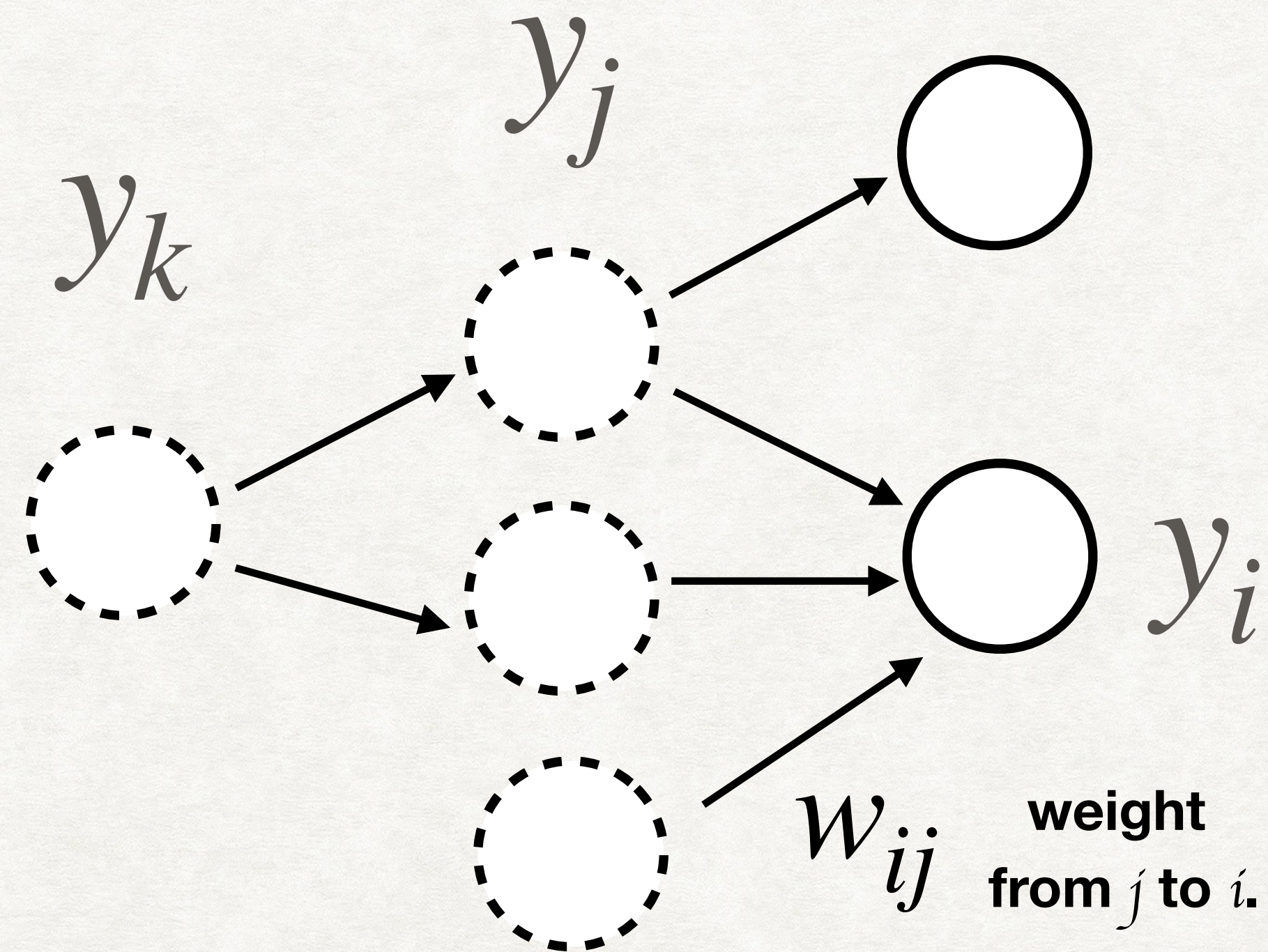
Sum over the batch dimension (axis=0) to get individual updates for each output neuron - in case there are more than one

**update_biases = -learning_rate * np.sum(error_times_derivative, axis=0) /
X.shape[0]**

f: sigmoid

ARTIFICIAL NEURAL NETWORKS AS FUNCTION APPROXIMATIONS

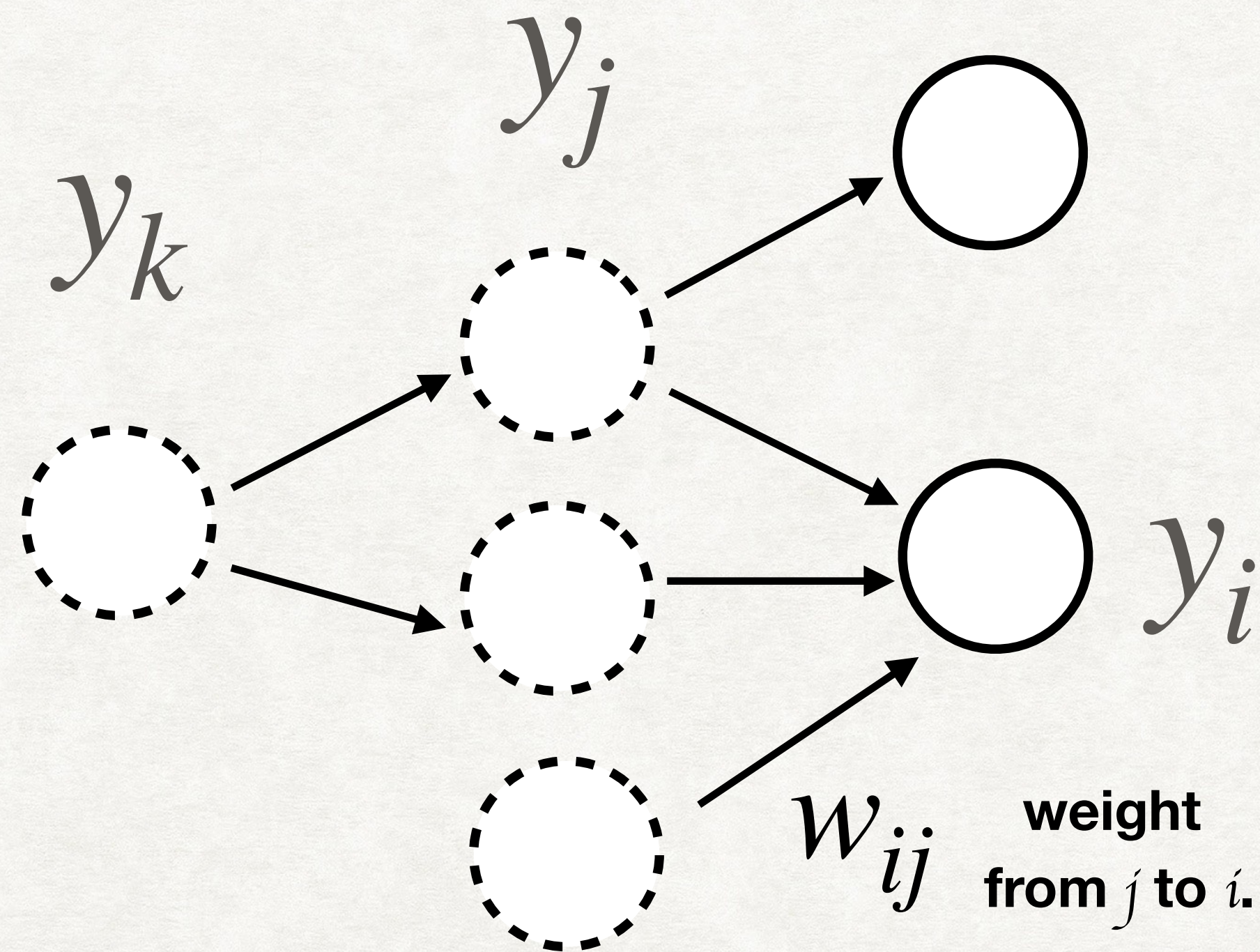
NON-LINEARLY SEPARABLE PROBLEMS



Many layers

ARTIFICIAL NEURAL NETWORKS AS FUNCTION APPROXIMATIONS

HIDDEN LAYER



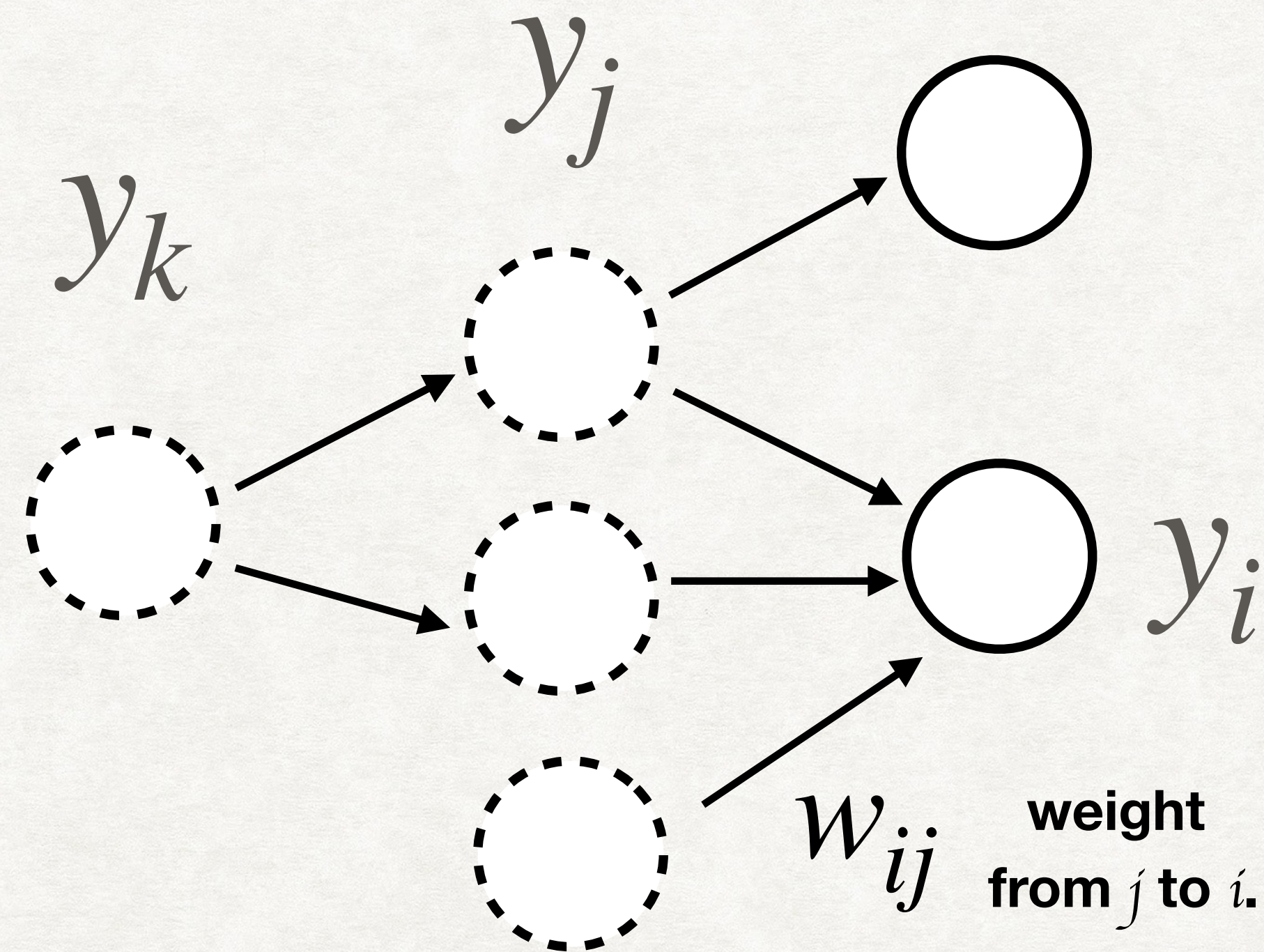
$$E = \frac{1}{2n} \sum_i \sum_{\mathbf{x}} (y_i^t(\mathbf{x}) - y_i(\mathbf{x}; \mathbf{W}))^2$$

$$\Delta w_{ij} = -\eta \frac{\partial E}{\partial w_{ij}}$$

$$\Delta w_{jk} = -\eta \frac{\partial E}{\partial w_{jk}}$$

ARTIFICIAL NEURAL NETWORKS AS FUNCTION APPROXIMATIONS

HIDDEN LAYER



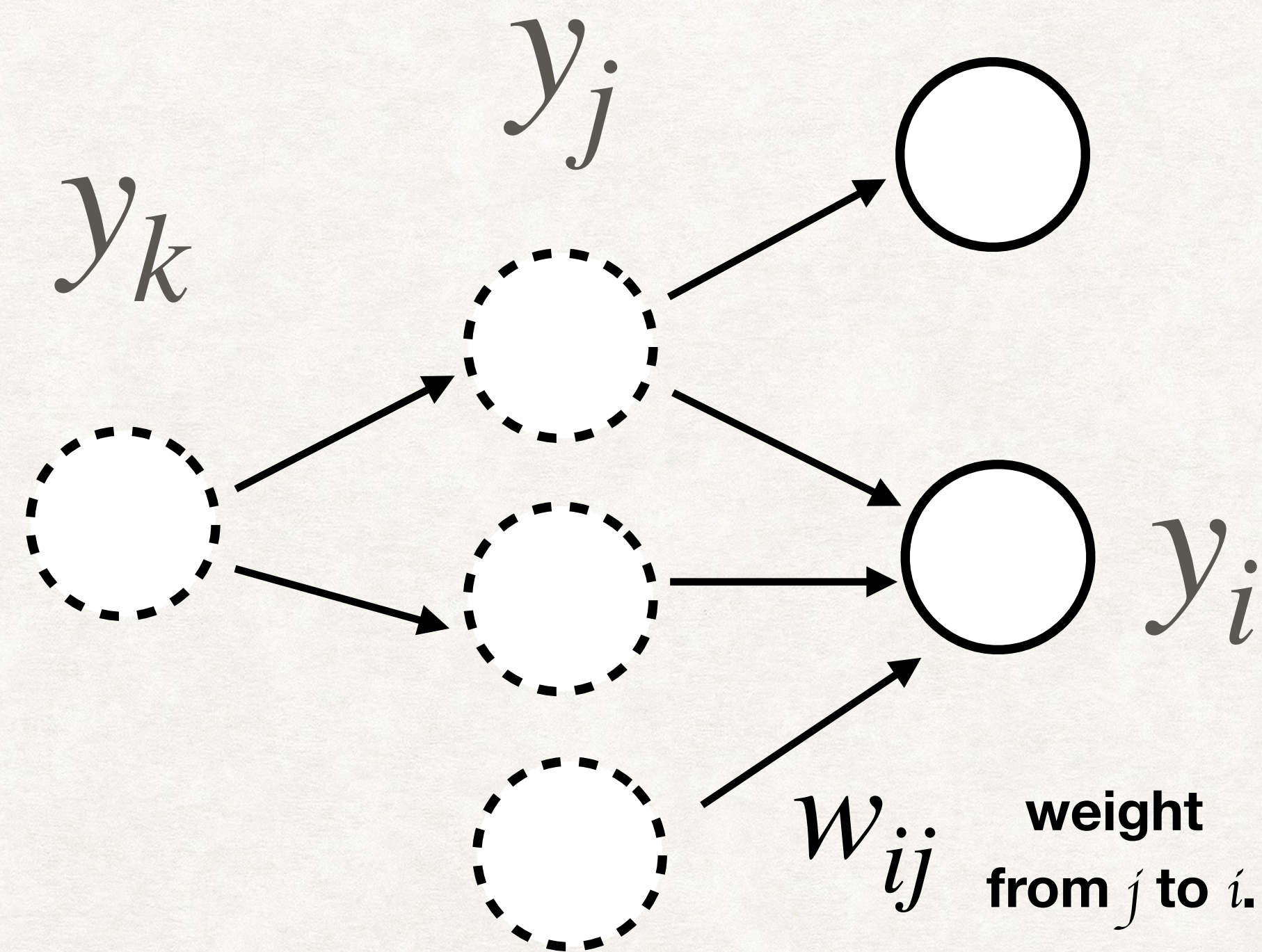
$$E = \frac{1}{2n} \sum_i \sum_{\mathbf{x}} (y_i^t(\mathbf{x}) - y_i(\mathbf{x}; \mathbf{W}))^2$$

$$\frac{\partial E}{\partial y_i} = \frac{1}{n} \sum_{\mathbf{x}} (y_i(\mathbf{x}; \mathbf{W}) - y_i^t(\mathbf{x}))$$

$$\frac{\partial E}{\partial y_j} = \sum_i \overset{\checkmark}{\frac{\partial E}{\partial y_i}} \frac{\partial y_i}{\partial y_j}$$

ARTIFICIAL NEURAL NETWORKS AS FUNCTION APPROXIMATIONS

HIDDEN LAYER



$$\frac{\partial E}{\partial y_j} = \sum_i \overset{\checkmark}{\frac{\partial E}{\partial y_i} \frac{\partial y_i}{\partial y_j}}$$

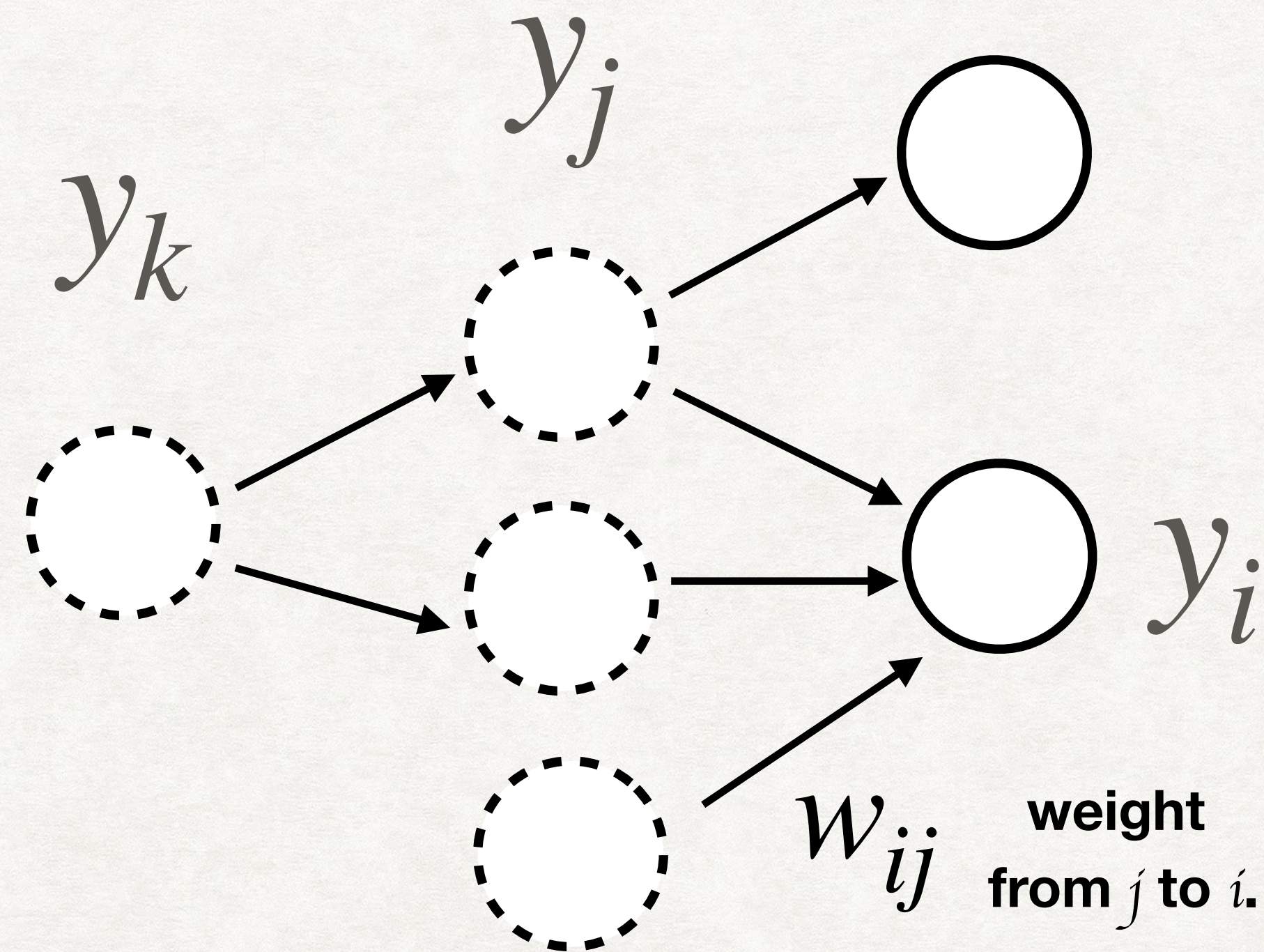
$$y_i = f\left(\sum_j w_{ij}y_j + b_i\right) = f(h_i)$$

$$\frac{\partial y_i}{\partial y_j} = f'(h_i)w_{ij}$$

$$\frac{\partial E}{\partial y_j} = \sum_i \frac{\partial E}{\partial y_i} f'(h_i)w_{ij}$$

ARTIFICIAL NEURAL NETWORKS AS FUNCTION APPROXIMATIONS

HIDDEN LAYER



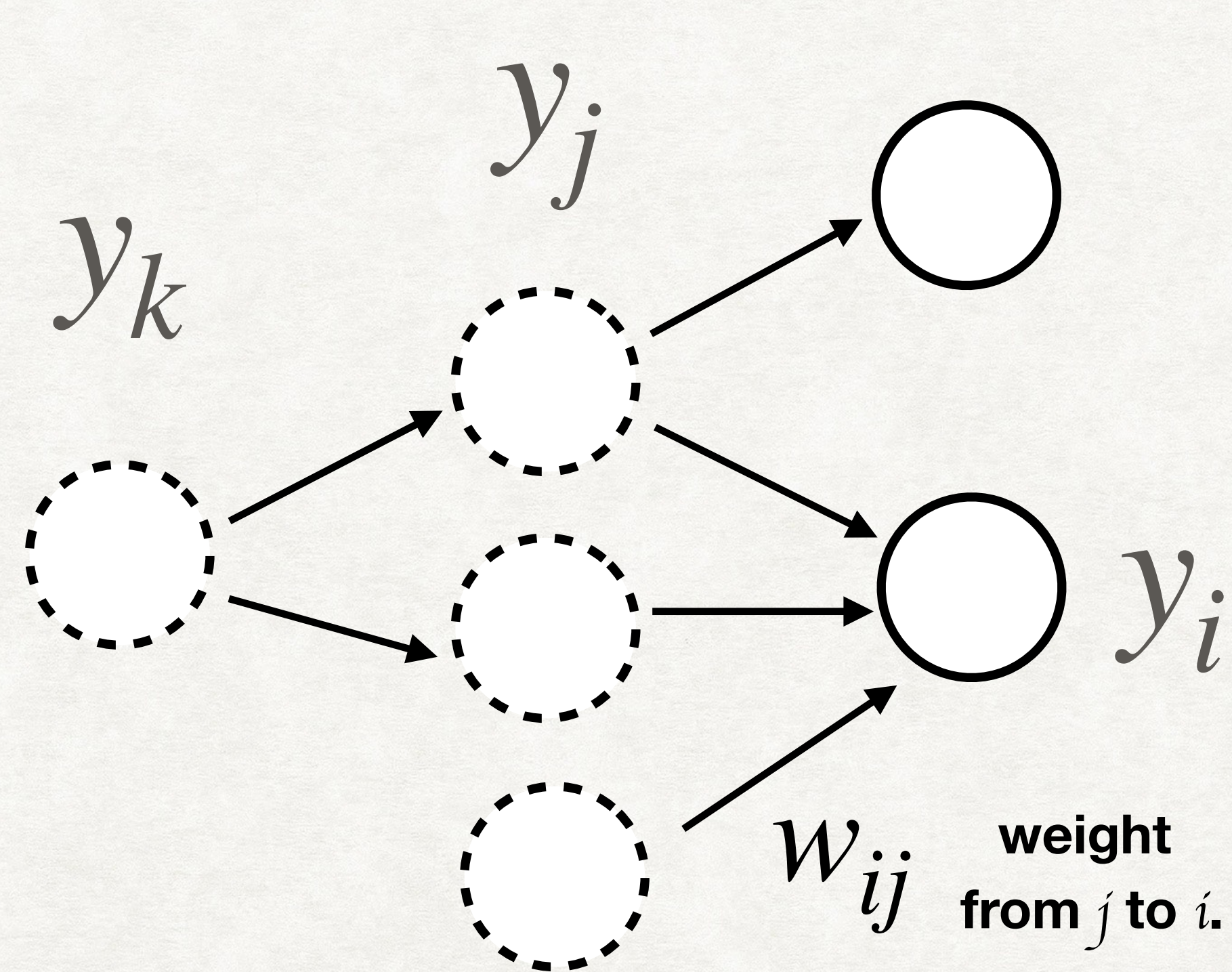
$$\frac{\partial E}{\partial y_j} = \sum_i \frac{\partial E}{\partial y_i} f'(h_i) w_{ij}$$

The “blame” of neuron y_j
wrt the error of neuron y_i
is through weight w_{ij} .

y_j contributes to the
error of all y_i neurons

ARTIFICIAL NEURAL NETWORKS AS FUNCTION APPROXIMATIONS

HIDDEN LAYER



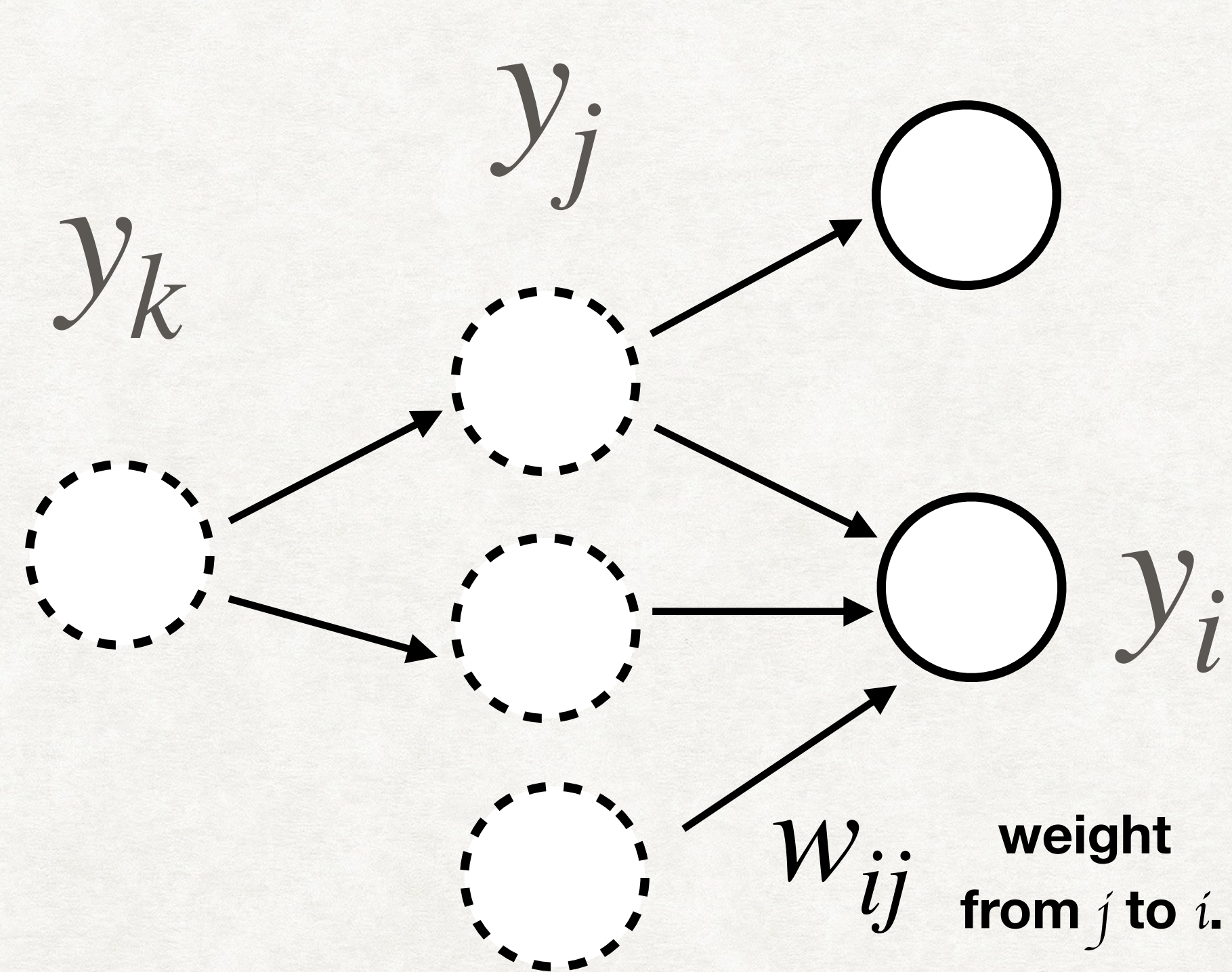
$$\frac{\partial E}{\partial w_{jk}} = \overset{\checkmark}{\frac{\partial E}{\partial y_j}} \frac{\partial y_j}{\partial w_{jk}}$$

$$y_j = f\left(\sum_k w_{jk} y_k + b_j\right) = f(h_j)$$

$$\frac{\partial E}{\partial w_{jk}} = \frac{\partial E}{\partial y_j} f'(h_j) y_k$$

ARTIFICIAL NEURAL NETWORKS AS FUNCTION APPROXIMATIONS

HIDDEN LAYER



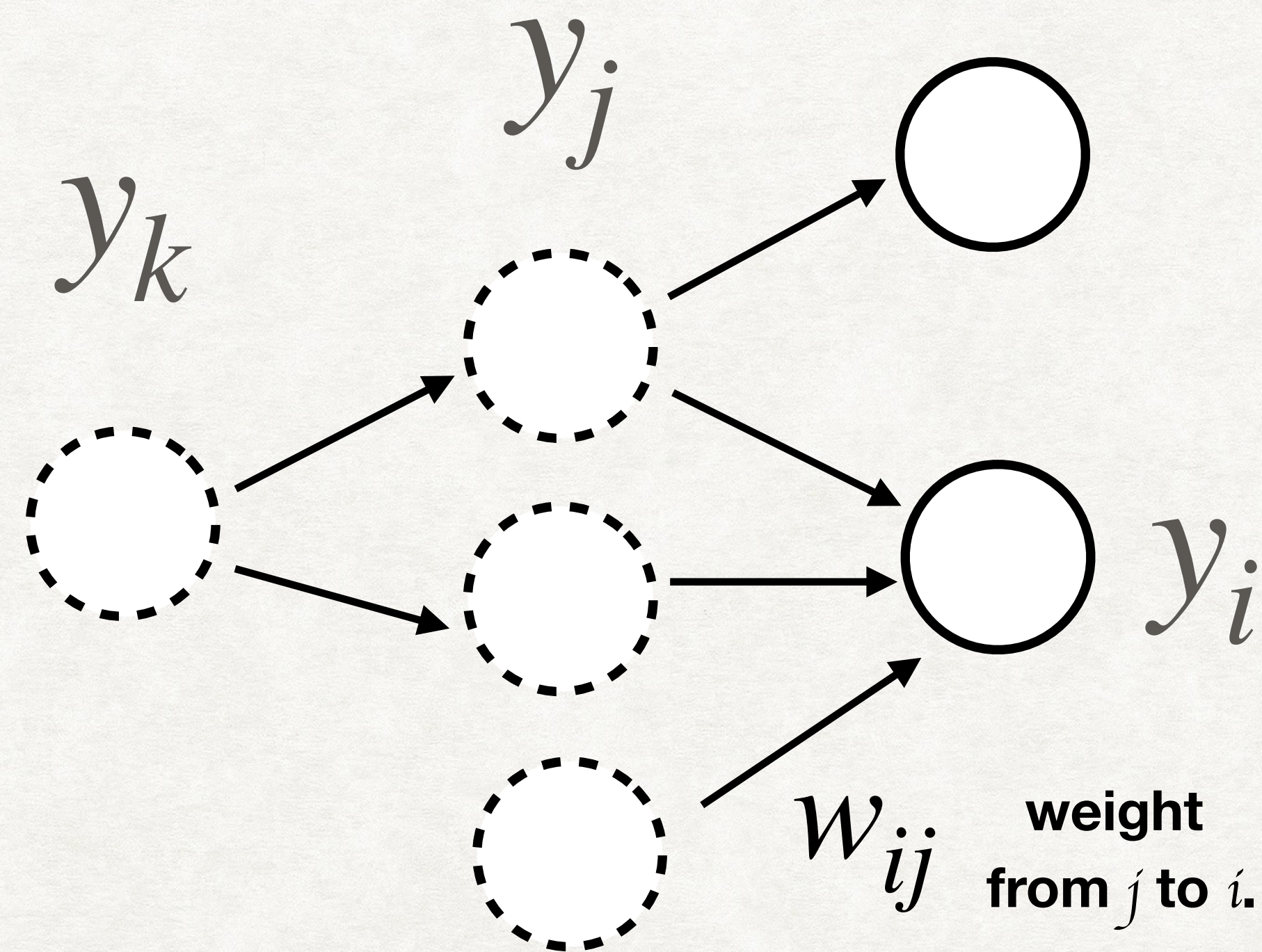
$$\frac{\partial E}{\partial w_{jk}} = \overset{\checkmark}{\frac{\partial E}{\partial y_j}} \frac{\partial y_j}{\partial w_{jk}}$$

$$\Delta w_{jk} = -\eta \frac{\partial E}{\partial w_{jk}} = -\eta \frac{\partial E}{\partial y_j} f'(h_j) y_k,$$

$$\Delta w_{jk} = -\eta \left(\sum_i \frac{\partial E}{\partial y_i} f'(h_i) w_{ij} \right) f'(h_j) y_k$$

ARTIFICIAL NEURAL NETWORKS AS FUNCTION APPROXIMATIONS

HIDDEN LAYER



$$\Delta w_{jk} = -\eta \left(\sum_i \frac{\partial E}{\partial y_i} f'(h_i) w_{ij} \right) f'(h_j) y_k$$

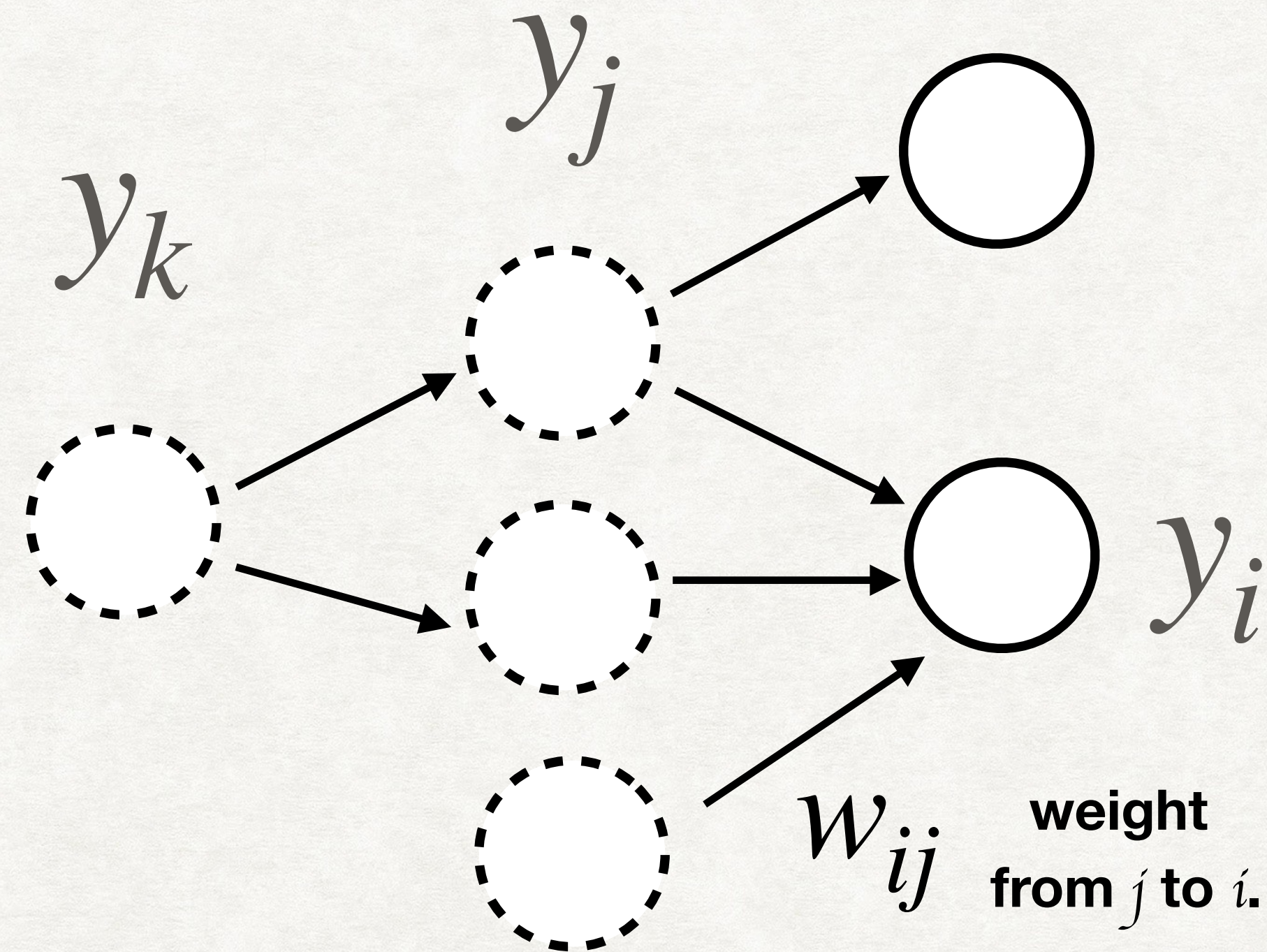
$$\delta_i = \frac{\partial E}{\partial y_i} f'(h_i)$$

$$\delta_j = \sum_i \delta_i w_{ij} f'(h_j)$$

$$\Delta w_{jk} = -\eta \delta_j y_k$$

ARTIFICIAL NEURAL NETWORKS AS FUNCTION APPROXIMATIONS

HIDDEN LAYER



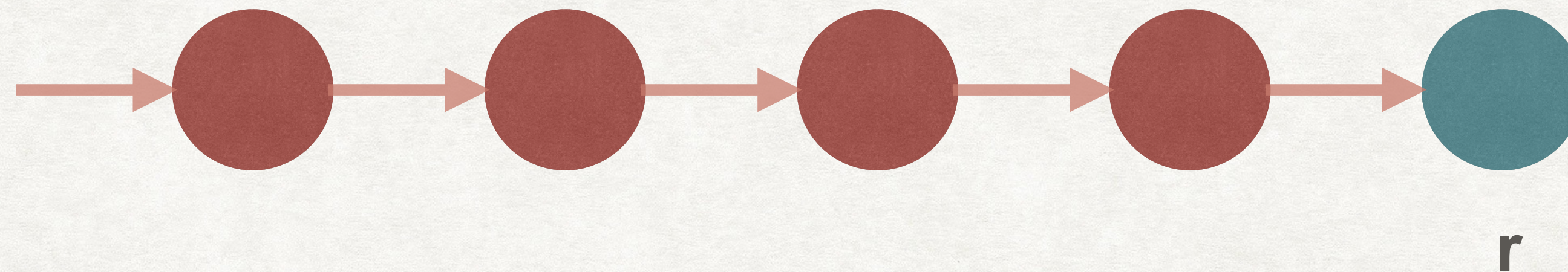
Forward pass: calculate all neuron activities and the error

Backword pass:
“backpropagate” error “blame”
based on the calculations of the
gradients, and update weights.

ARTIFICIAL NEURAL NETWORKS AS FUNCTION APPROXIMATIONS

REPLACING THE Q-VALUE TABLE WITH A NEURAL NETWORK

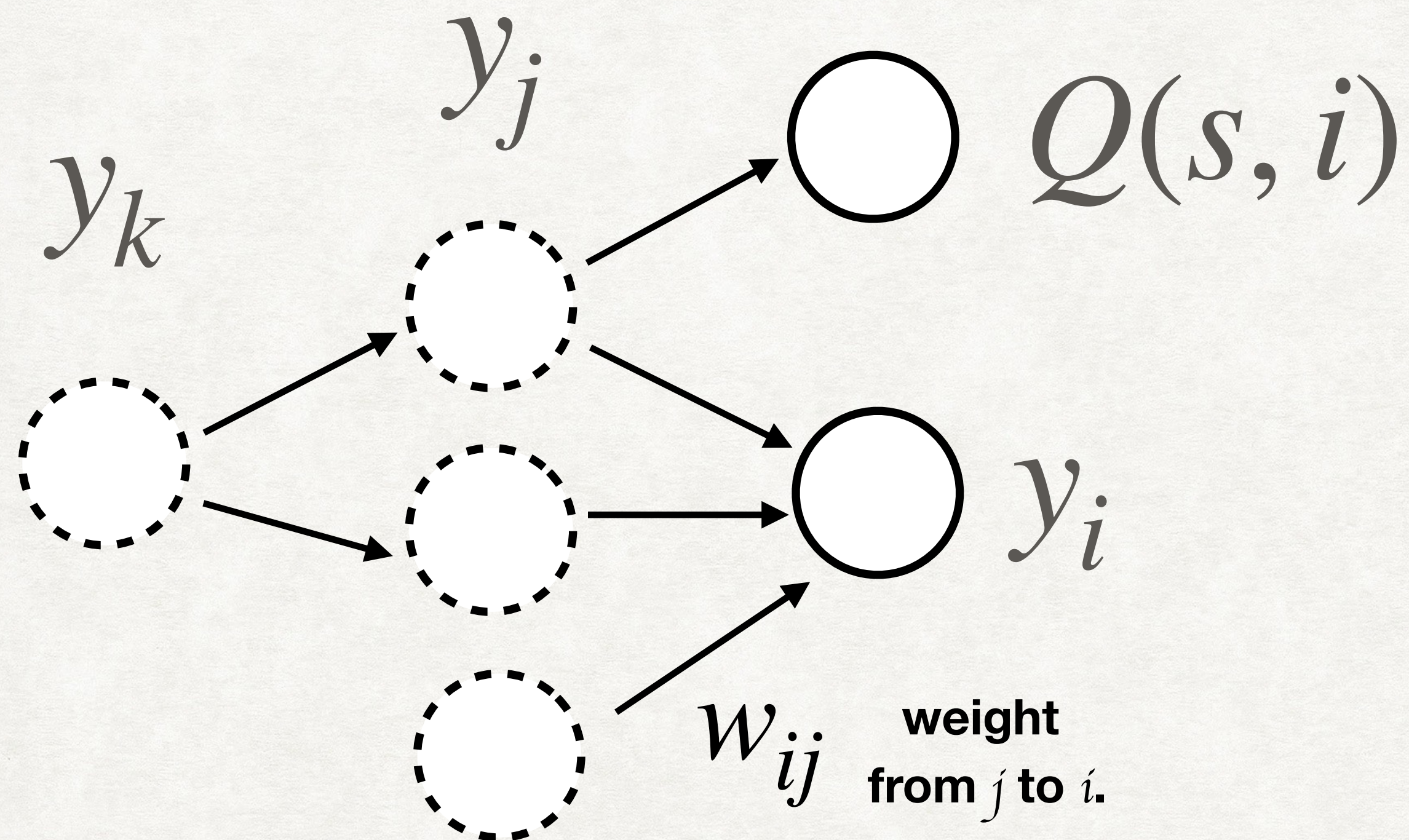
$$\Delta Q = \eta (r + \gamma Q(s', a') - Q(s, a))$$



A robot walks this corridor and finds reward.

ARTIFICIAL NEURAL NETWORKS AS FUNCTION APPROXIMATIONS

REPLACING THE Q-VALUE TABLE WITH A NEURAL NETWORK



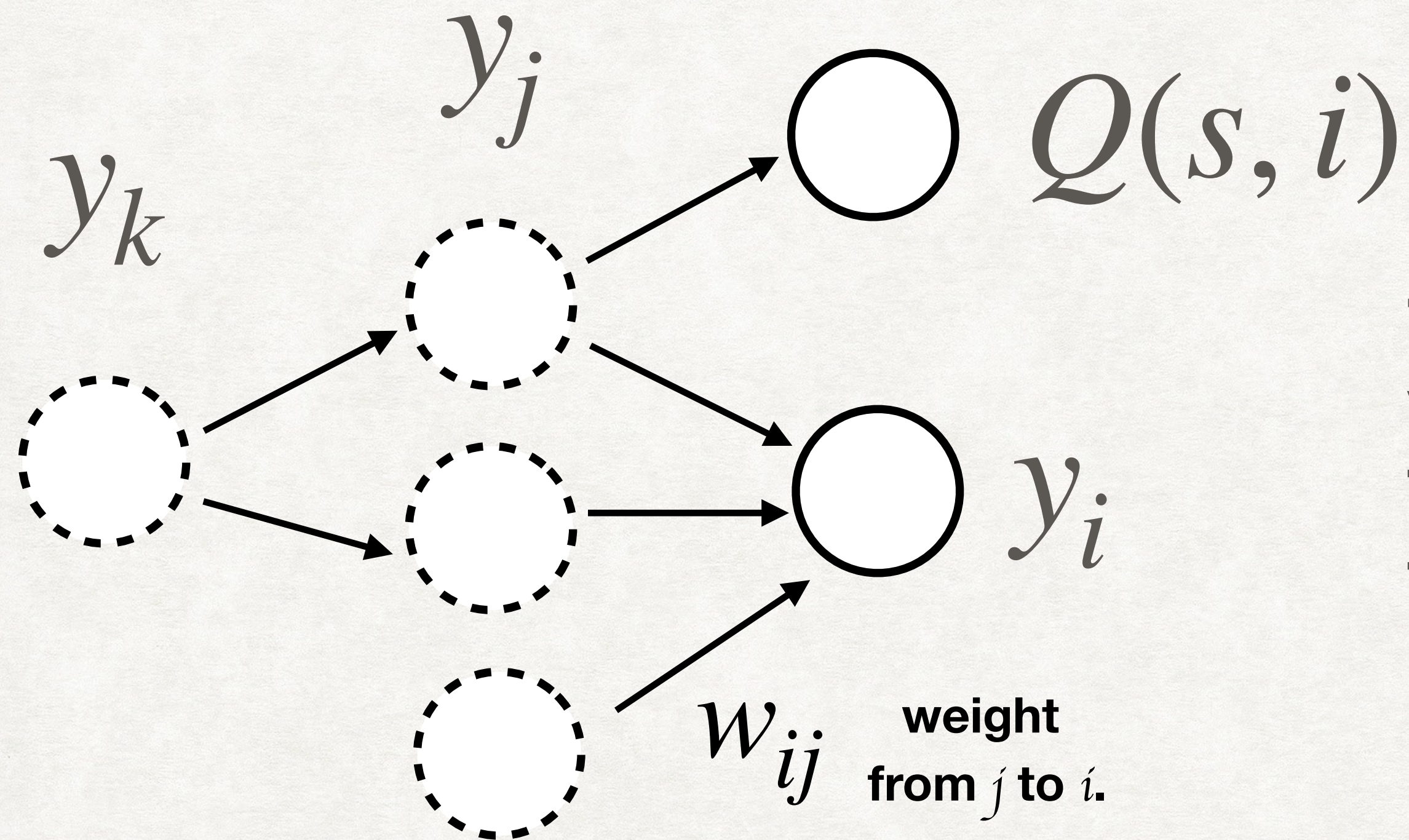
I only want to update the pathways to the neuron corresponding to the action taken.

Input=State s

$$y_i^t(s) = r + \gamma \max_{i'} Q(s', i') = r + \gamma \max_{a'} Q(s', a')$$

ARTIFICIAL NEURAL NETWORKS AS FUNCTION APPROXIMATIONS

REPLACING THE Q-VALUE TABLE WITH A NEURAL NETWORK



Trick: I construct my target as the actual values for all output neurons apart from the one corresponding to the action taken. Only this one is set to $y_i^t(s)$.

Input=State s

$$y_i^t(s) = r + \gamma \max_{i'} Q(s', i') = r + \gamma \max_{a'} Q(s', a')$$

ARTIFICIAL NEURAL NETWORKS AS FUNCTION APPROXIMATIONS

REPLACING THE Q-VALUE TABLE WITH A NEURAL NETWORK



DALL·E

THANK YOU!